DISTRIBUTED APPLICATION DEVELOPMENT — TERM PROJECT

Ring of the Middle Earth

Architecture Document

OPTION
Option B — Go + Kafka

TECHNOLOGY
KTable State Stores

COURSE
Distributed App Development

EGE UNIVERSITESI · MUHENDISLIK FAKULTESI · BILGISAYAR MUHENDISLIGI

1. System Diagram — Services, Connections, and Data Flow

1.1 Service Overview

The system runs entirely in Docker. All services are declared in `docker-compose.yml`. There are **10 persistent services**; 2 initialisation containers exit after startup.

Service	Type	Port(s)	Notes
nginx	Edge / Reverse Proxy	80	Round-robin upstream: go-1 | go-2 | go-3. Serves static UI.
go-1, go-2, go-3	Game Engine	8080 (pprof :6060)	Consumer group: game-engine-*. Identical instances.
kafka-1/2/3	Message Broker	9092/9093/9094	replication-factor=3, min.insync.replicas=2
schema-registry	Schema Registry	8081	Avro/JSON schema management
streams-processor	Stream Processor	—	Runs Topology 1 (streams-t1) and Topology 2 (streams-t2)
zookeeper	Coordination	2181	Kafka cluster coordination
kafka-init	Init (exits)	—	Topic setup — terminates after completion
schema-init	Init (exits)	—	Schema registration — terminates after completion

1.2 Request / Event Data Flow

1. Player submits an order via HTTP POST /order — nginx load-balances to any Go node.
2. Go Engine produces the order to game.orders.raw.
3. Topology 1 (streams-t1) consumes game.orders.raw and validates it against KTables (TurnKTable, PlayerKTable, UnitKTable, PathKTable, RegionKTable).

4. Valid orders → game.orders.validated (also consumed by Topology 2 for route-risk enrichment). Invalid orders → game.dlq.
5. Go Engine consumes game.orders.validated, executes turn logic, and produces events to: game.events.unit, game.events.region, game.events.path, game.ring.position (Light Side SSE only), game.ring.detection (Dark Side SSE only), game.session, game.broadcast.

2. Option B: Goroutine Map

Each of the three Go engine instances (go-1, go-2, go-3) runs the same set of goroutines. The map below describes **one instance**. Goroutines are terminated by `ctx.Done()` unless noted otherwise.

Go Engine Instance (go-N)

ID	Name / Role	Input Channel	Output Channel	Termination
[G0]	main — Wires all channels, builds CacheManager, EventRouter, HTTP Server. Calls recoverState() synchronously.	–	–	SIGTERM → cancel() → all children exit
[G1]	consumer — confluent-kafka-go poll loop across 8 topics.	Kafka topics (8)	eventCh (unbuffered)	ctx.Done()
[G2]	cacheManager — mutates internal cache under sync.RWMutex. Invariant: DarkView.RingBearerRegion = ""	updateCh (cap=16)	none (mutates cache)	ctx.Done() or updateCh closed
[G3]	eventRouter — select loop, 7 cases. Enforces info asymmetry: ring.position → Light only; ring.detection → Dark only.	eventCh (unbuffered)	engineCh (8), lightSideSSECh (32), darkSideSSECh (32), cacheUpdateCh (16)	ctx.Done()
[G4]	gameEngine — reads validated orders, executes 13-step turn logic. Idempotent producer. GameOver uses ProduceSync().	engineCh (cap=8)	Kafka topics	ctx.Done()

ID	Name / Role	Input Channel	Output Channel	Termination
[G5]	httpServer — net/http mux. Routes: GET /sse, POST /order, GET /state.	—	Spawns G6 per connection	server.Shutdown(ctx)
[G6]	handleSSE — per-player SSE goroutine. Light Side → lightSideSSECh; Dark Side → darkSideSSECh.	writeCh (cap=32)	HTTP text/event-stream	r.Context().Done() → sends playerID to disconnectCh
[G7-G10]	RouteRisk workers — transient, 4 per invocation. Fan-out/fan-in. Timeout: 2s; partial results on expiry.	workCh (cap=20)	resultCh (unbuffered)	workCh closed or ctx.Done()
[G11-GN]	Intercept workers — transient, count=len(jobs). Same fan-out/fan-in pattern. Timeout: 2s.	workCh (cap=20)	resultCh (unbuffered)	workCh closed or ctx.Done()

streams-processor (separate binary — kafka/streams/)

ID	Role	Subscribes / Produces	Termination
[S0]	main — Wires runner, waits on SIGTERM.	—	SIGTERM
[S1]	Topology1 — Group: streams-t1. Maintains TurnKTable, PlayerKTable, UnitKTable, PathKTable, RegionKTable.	Subscribes: game.orders.raw, game.session, game.events.* Produces: game.orders.validated, game.dlq	ctx.Done()
[S2]	Topology2 — Group: streams-t2. Enriches route-risk orders with current map state.	Subscribes: game.orders.validated, game.events.* Produces: enriched → game.orders.validated	ctx.Done()

ID	Role	Subscribes / Produces	Termination
[S3]	producer — Delivery report goroutine (confluent-kafka-go internal).	–	—

3. Kafka Diagram — Topics, Producers, Consumers, Partition Key

Topic	Parts	Cleanup / Retention	Producer(s)	Consumer(s)	Part. Key	Rationale
`game.orders.raw`	3	delete · 1h	Go engine (any)	streams-t1	`playerID`	All orders from same player on one partition — preserves per-player ordering for turn validation.
`game.orders.validated`	6	delete · 1h	streams-processor (T1&T2;)	Go engine; streams-t2	`playerID`	Ordering guarantee maintained; any Go node on same partition sees full ordered history per player.
`game.events.unit`	6	delete · 7d	Go engine	Go engine; streams-t1; streams-t2	`unitID`	All state updates for a unit co-located on one partition. KTable rebuilds per-unit state from single partition.
`game.events.region`	6	delete · 7d	Go engine	Go engine; streams-t1; streams-t2	`regionID`	Region state partitioned by regionID for locality and ordered replay.
`game.events.path`	6	delete · 7d	Go engine	Go engine; streams-t1; streams-t2	`pathID`	Per-path ordering ensures status transitions (OPEN→BLOCKED→OPEN) applied in arrival order within KTable.
`game.session`	1	compact · inf	Go engine (snapshots)	Go engine (recovery); streams-t1	`recordtype`	Single partition + log compaction = KTable. Latest record per key retained forever. Total ordering for session state.
`game.broadcast`	1	delete · 1h	Go engine (ProduceSync)	Go engine (both SSE sides)	`eventtype`	Single partition: all nodes receive broadcasts in same global order. GameOver uses ProduceSync.

Topic	Parts	Cleanup / Retention	Producer(s)	Consumer(s)	Part. Key	Rationale
`game.ring.position`	1	delete · 1h	Go engine	Go engine → Light Side SSE	`sessionID`	Ring Bearer position is singleton. Strict ordering — position events delivered in turn order to Light Side.
`game.ring.detection`	2	delete · 1h	Go engine	Go engine → Dark Side SSE	`nazgulID`	Two partitions allow two Nazgul streams in parallel while maintaining per-Nazgul ordering.
`game.dlq`	3	delete · 7d	streams-processor (T1)	Monitoring / replay tools	`playerID`	Failed orders keyed by playerID for easy filtering. 7-day retention allows post-game audit.

3.1 Topic Flow

```
Go Engine → game.orders.raw → Topology 1 (validates against KTables)
|-- Valid → game.orders.validated → Go Engine (turn logic) + Topology 2 (route enrichment)
\-- Invalid → game.dlq

Go Engine produces after turn logic:
game.events.unit / region / path → KTable feeds (T1/T2) + cache updates
game.ring.position → Light Side SSE only
game.ring.detection → Dark Side SSE only
game.session → state recovery + TurnKTable, PlayerKTable
game.broadcast → both SSE sides
```

4. Paradigm Justification

4.1 Why is the CSP / Goroutine paradigm well-suited to this problem?

The game has a fundamentally concurrent structure: multiple browser clients connect simultaneously, orders arrive asynchronously from either side, Kafka delivers events from multiple partitions at once, and SSE streams must push updates to players without blocking order processing.

Go's CSP model — goroutines communicating over typed channels — maps directly onto this structure. Each logical role (Kafka consumer, cache manager, event router, game engine, per-player SSE writer) becomes an independent goroutine with a clearly defined input and output channel. There is no shared mutable state between goroutines; all coordination is message-passing. This makes the data flow readable as a directed graph of channels.

The information asymmetry requirement is naturally expressed as two separate SSE channels (lightSideSSECh, darkSideSSECh). The EventRouter goroutine is the single enforcement point — this constraint is structural, not documentary, and cannot be bypassed accidentally.

The fan-out/fan-in pipeline pattern for route risk and intercept scoring is another natural fit. Spawning 4 worker goroutines per pipeline invocation and collecting results through an aggregator is idiomatic Go. A context.WithTimeout provides graceful degradation — partial results on expiry rather than blocking.

4.2 What is genuinely harder with CSP than with the Actor model?

State recovery and KTable reconstruction. In Akka (Option A), each actor has a private mailbox and persistent state; supervision strategies handle restarts automatically. In Go/Kafka there is no built-in supervision tree. When a Go node restarts it must re-subscribe (triggering partition rebalancing) and replay game.session from offset 0 to reconstruct in-memory world state. The RecoverStateFromKafka function implements this manually — verbose, fragile to clock-based deadlines, and must re-apply the invariant (DarkView.RingBearerRegion = "") after every recovery.

Channel lifecycle and goroutine leak prevention. In the Actor model the framework owns every actor's lifecycle. In Go every goroutine must have a clean termination path. The per-player SSE goroutine (G6) terminates only when the HTTP request context is

cancelled. If the server shuts down first, server.Shutdown(ctx) must drain in-flight requests and close their contexts — careful plumbing the Actor model handles via lifecycle hooks.

4.3 How would the Actor model (Option A) solve these two hard parts?

Hard Part 1 — State Recovery

In Option A, state recovery would use **Akka Persistence**. Each UnitActor, PathActor, and a central GameStateActor would persist their state as an event journal (Cassandra or JDBC). On restart the actor system replays the journal automatically during receiveRecover — no manual Kafka replay or temporary consumer group is needed. Supervision strategy: Resume for transient errors, Restart (triggering journal replay) for state corruption.

Hard Part 2 — Lifecycle / Supervision

Each per-player session would be a PlayerSessionActor under a GameSupervisor. If a player disconnects, the actor simply stops. The supervisor detects termination via DeathWatch and cleans up session state. No goroutine leaks are possible because the framework manages every actor's lifecycle. A dedicated SSEActor per player terminates cleanly on connection close, supervised with an Escalate strategy to the cluster singleton.

5. Reflection

5.1 What Was Harder Than Expected?

Information asymmetry constraint. The requirement that Dark Side players must never see the Ring Bearer's true position has a silent failure mode — if violated, a Dark Side SSE stream simply receives a JSON field it should not. No compile-time error, no runtime panic. A three-layer defence was implemented: (1) EventRouter.route() is the sole dispatch point; (2) CacheManager.Update() unconditionally zeroes DarkView.RingBearerRegion under write lock; (3) deepCopy() applies the same zeroing on every outbound snapshot. Running go test -race revealed two race points which were fixed by routing all reads through CacheManager.Get().

Avro schema evolution. The topic game.orders.validated is shared between order-validated-v1.avsc and order-validated-v2.avsc. Schema Registry requires both schemas under the same subject (game.orders.validated-value) with backward compatibility. Determining which fields could be added, which must keep defaults, and how the deserialiser resolves the correct version from the five-byte magic header required careful study of the Schema Registry wire format specification.

Exactly-once delivery for GameOver. Enabling enable.idempotence=true prevents duplicate produces from retries but does not prevent re-processing after partition rebalance. After a rebalance, a newly-assigned node could replay game.broadcast from a committed offset preceding the GameOver record and re-broadcast to SSE clients. Solution: the game engine checks cache.GameOver before re-broadcasting and suppresses duplicate SSE pushes within a session.

5.2 What Would Be Designed Differently?

Split game.session into three topics. Currently the single game.session topic serves three distinct KTable roles (TurnKTable, PlayerKTable, world-state snapshots) distinguished only by key prefix. This multiplexing violates single-responsibility for topics and makes the schema ambiguous. Three separate topics with explicit schemas would make each independently evolvable and monitored.

Secondary index for recovery. The current approach reads game.session from offset 0 on every restart, scanning for the latest world-state record with a 10-second deadline — scan time grows linearly with turns. A better design: a game.session.index topic storing only the most recent snapshot offset, so recovery seeks directly to that offset.

Long-lived worker pool. RouteRiskPipeline and InterceptPipeline allocate and GC 4–N goroutines on every invocation. For a 60-second turn duration this is acceptable, but for a higher-frequency system a persistent worker pool fed from a shared queue would eliminate repeated goroutine creation overhead and statically bound the maximum concurrency.